AUTOMATION FOR JAVA DEVELOPERS · PLAYWRIGHT

Playwright with Java: From Zero to Real Tests

You already know Java. This guide teaches only what is new: Playwright words, objects and methods, and how to use them to test a real web page. Every code example is Java and was run successfully.

IMPORTANT

Red boxes and rows: learn these first. They are used in almost every test and asked in almost every interview.

GOOD TO KNOW

Blue boxes and rows: useful, but you can learn them later or only when needed.

---

1. What is Playwright and how does it work?

Playwright is a Java library, made by Microsoft, that controls a real browser (Chromium, Firefox or WebKit). Your Java code says "open this page", "fill this box", "click that button". Then your code checks the result.

Compared with Selenium, Playwright **waits automatically** before every action, so you write far fewer waits. It also has built-in screenshots, video, tracing and network control.

Your Java test → **Playwright library** → **Playwright driver (Node, hidden inside the jar)** → **Browser**

You never see the middle part. When you run your Java code, Playwright starts it by itself.

The five objects you must understand

IMPORTANT **Object hierarchy**

OBJECT	WHAT IT IS	REAL LIFE PICTURE
`Playwright`	The starting point. Created once.	The power switch
`Browser`	One browser program (Chrome).	The browser application

`BrowserContext`	A clean, isolated browser profile with its own cookies and storage.	One incognito window
`Page`	One tab inside a context. Almost all your code uses `page`.	One tab
`Locator`	A way to find an element on the page. It finds the element again every time you use it.	A description: "the Login button"

The chain is: `Playwright` → `Browser` → `BrowserContext` → `Page` → `Locator`. The best habit is **one new context per test**. It is very cheap and every test starts clean, with no cookies left from the last test.

Words you will hear

WORD	SIMPLE MEANING
Auto-waiting	Before clicking, Playwright waits until the element is ready. You do not write `Thread.sleep` or explicit waits.
Web-first assertions	Assertions like `assertThat(locator).hasText(..)` retry for a few seconds until true. This kills flaky tests.
Headless / headed	Headless = no window. Headed = you can watch the browser.
Channel	Which installed browser to use. `setChannel("chrome")` uses your normal Google Chrome.
Trace	A recording of the test (steps, screenshots, network). You open it later to see what went wrong.
Codegen	A tool that records your clicks and writes Java code for you.
Inspector	A debugging window that opens when you call `page.pause()`.
Strict mode	If a locator matches more than one element, an action on it throws an error. Prevents clicking the wrong one.

2. Setup in Java (Maven)

IMPORTANT **Dependencies**

Add this to the `dependencies` of your `pom.xml`, plus TestNG for running tests.

```
<dependency>
  <groupId>com.microsoft.playwright</groupId>
  <artifactId>playwright</artifactId>
  <version>1.49.0</version>
</dependency>
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.10.2</version>
</dependency>
```

What it does: One dependency is enough for Playwright. You can use a newer version from Maven Central.

The first time you run Playwright, it downloads its own browsers (a few hundred MB). To use the Chrome already on your computer, add `.setChannel("chrome")` when launching, as the examples below do. To download the browsers by hand:

```
mvn exec:java -e -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="insta
```

What it does: Installs Chromium, Firefox and WebKit for Playwright.

Your first test (complete)

```
import com.microsoft.playwright.*;

public class FirstTest {
    public static void main(String[] args) {
        try (Playwright playwright = Playwright.create()) {                  // 1.
            Browser browser = playwright.chromium().launch(
                    new BrowserType.LaunchOptions().setHeadless(false));     // 2.
            Page page = browser.newPage();                                   // 3.
            page.navigate("https://example.com");                            // 4.
            System.out.println(page.title());                                // 5.
            page.locator("a").click();                                       // 6.
            browser.close();                                                 // 7.
        }
    }
}
```

What it does: The same five steps as in any browser test: open, navigate, act, check, close. The `try (...)` block closes Playwright for you at the end.

> **IMPORTANT** **Cross-browser in one line**
>
> `playwright.chromium()` can also be `firefox()` or `webkit()`. The rest of the code stays the same. That is how you test on three browser engines.

---

3. The core objects and their methods

Playwright and BrowserType

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`Playwright.create()`	Starts Playwright. Always close it at the end (`try-with-resources` or `playwright.close()`).	`Playwright pw = Playwright.create();`
IMPORTANT	`playwright.chromium() / firefox() / webkit()`	Choose a browser engine.	`BrowserType bt = playwright.chromium();`

IMPORTANT	`browserType.launch(LaunchOptions)`	Opens the browser and returns a `Browser`.	`bt.launch(new BrowserType.LaunchOptions().setHeadless(false));`
IMPORTANT	`LaunchOptions.setHeadless(boolean)`	false = show the window. Default is true (no window).	
IMPORTANT	`LaunchOptions.setChannel("chrome")`	Use the installed Google Chrome (or "msedge").	
GOOD TO KNOW	`LaunchOptions.setSlowMo(500)`	Slows every action by 500 ms so you can watch.	
GOOD TO KNOW	`LaunchOptions.setArgs(List.of("--start-maximized"))`	Pass extra browser arguments.	

Browser

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`browser.newContext()`	Creates a clean `BrowserContext`.	`BrowserContext ctx = browser.newContext();`
IMPORTANT	`browser.newPage()`	Shortcut: creates a new context AND a page in one go.	`Page page = browser.newPage();`
IMPORTANT	`browser.close()`	Closes the browser and all its pages.	
GOOD TO KNOW	`browser.newContext(new Browser.NewContextOptions()...)`	Options: `setViewportSize(1280, 720)`, `setLocale("en-IN")`, `setGeolocation(...)`, `setRecordVideoDir(path)`, `setStorageState(path)`.	

BrowserContext

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`context.newPage()`	Opens a new tab.	`Page page = context.newPage();`
IMPORTANT	`context.close()`	Closes this context and its pages.	
GOOD TO KNOW	`context.cookies()` / `addCookies(...)` / `clearCookies()`	Read, add, or clear cookies.	
GOOD TO KNOW	`context.storageState(...)`	Saves login state (cookies + storage) to a file so other tests can skip the login.	
GOOD TO KNOW	`context.tracing().start(...)` / `stop(...)`	Records a trace of the test. See the debugging section.	
GOOD TO KNOW	`context.setDefaultTimeout(ms)`	Default wait time for every action in this context.	
GOOD TO KNOW	`context.route(...)`	Intercept or fake network calls for the whole context.	

Page (the tab)

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`page.navigate(String url)`	Opens a URL and waits for it to load.	`page.navigate("https://shop.com");`
IMPORTANT	`page.title()`	Returns the page title.	`String t = page.title();`
IMPORTANT	`page.url()`	Returns the current URL.	
IMPORTANT	`page.locator(String selector)`	Creates a Locator from CSS or XPath.	`page.locator("#email")`

IMPORTANT	`page.getByRole / getByLabel / getByText / ...`	Creates a Locator the way a user sees the page. See the locators section.	
IMPORTANT	`page.reload() / goBack() / goForward()`	Browser reload, back and forward.	`page.goBack();`
IMPORTANT	`page.onDialog(handler) / onceDialog(handler)`	Handle alert, confirm and prompt popups. Register BEFORE the action that opens them.	
IMPORTANT	`page.screenshot(...)`	Takes a screenshot.	`page.screenshot(new Page.ScreenshotOptions().setPath(Paths.get("a.png")).setFullPage(true));`
IMPORTANT	`page.close()`	Closes the tab.	
GOOD TO KNOW	`page.waitForLoadState() / waitForURL(...)`	Wait for the page to finish loading or for the URL to change. Rarely needed because of auto-wait.	
GOOD TO KNOW	`page.waitForPopup(Runnable) / context.waitForPage(...)`	Get the new tab that an action opens.	
GOOD TO KNOW	`page.waitForDownload(Runnable)`	Get the file that an action downloads.	
GOOD TO KNOW	`page.frameLocator(String)`	Work inside an iframe.	
GOOD TO KNOW	`page.evaluate(String js)`	Run JavaScript in the page and get the result.	`page.evaluate("document.title")`
GOOD TO KNOW	`page.pause()`	Opens the Inspector so you can debug step by step (needs headed mode).	

GOOD TO KNOW	`page.setViewportSize(w, h)`	Change the window size, for example a phone size.	`page.setViewportSize(375, 812);`
GOOD TO KNOW	`page.content()`	The full HTML of the page.	
GOOD TO KNOW	`page.keyboard()` / `page.mouse()`	Low level keys and mouse: `keyboard().press("Control+A")`, `mouse().click(x, y)`.	
GOOD TO KNOW	`page.route(...)` / `waitForResponse(...)`	Fake, block or watch network requests.	
GOOD TO KNOW	`page.setDefaultTimeout(ms)`	Default wait time for this page.	`page.setDefaultTimeout(10000);`

4. Locators: how to find an element

A **Locator** describes an element. It is lazy: nothing is searched until you act on it, and it searches again on every action. So it never becomes "stale" like a Selenium element can.

IMPORTANT **Best order of choice**

Playwright prefers locators that match **how a user sees the page**: by role (button, link, textbox), by label, by text. These survive HTML changes better than ids and XPath. Use CSS or XPath when there is no better choice.

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Login"))`	Finds by role and its visible name. Best for buttons, links, headings, checkboxes.	
IMPORTANT	`page.getByLabel("Email")`	Finds an input by its `<label>` text. Best for forms.	
IMPORTANT	`page.getByPlaceholder("Search products")`	Finds an input by its placeholder.	

IMPORTANT	`page.getByText("Welcome")`	Finds by visible text (partial, ignores case by default). Add `setExact(true)` for exact match.
IMPORTANT	`page.locator("#email")`	CSS selector (or XPath: `page.locator("//input[@id='email']")`).
IMPORTANT	`page.getByTestId("submit")`	Finds by the `data-testid` attribute. Developers add this just for testing. Most stable.
GOOD TO KNOW	`page.getByAltText("Logo")` / `getByTitle("Close")`	Find an image by alt text, or an element by title.

Refining a locator

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`locator.locator("button")`	Search inside an element (chaining).	`page.locator("#product-3").locator("button")`
IMPORTANT	`locator.first()` / `last()` / `nth(2)`	Pick one of many matches. `nth` starts at 0.	`page.locator(".product").nth(1)`
IMPORTANT	`locator.count()`	How many elements match. Returns an `int`.	`page.locator(".product").count()`
GOOD TO KNOW	`locator.filter(new Locator.FilterOptions().setHasText("Pen"))`	Keep only the matches that contain some text.	
GOOD TO KNOW	`locator.and(other)` / `locator.or(other)`	Combine two locators.	
GOOD TO KNOW	`locator.all()`	Returns a `List<Locator>` to loop over.	`for (Locator p : page.locator(".product").all()) { ... }`

IMPORTANT **Two things that surprise beginners**

- **Strict mode:** if `page.locator(".product").click()` matches 3 elements, Playwright throws an error instead of clicking one by chance. Fix it with a more exact locator, or `.first()`, or `.nth(i)`.
- `getByLabel("Password")` also matches "Confirm password", because text matching is partial. Use `new Page.GetByLabelOptions().setExact(true)` to match exactly.
- Test locators quickly with `npx playwright codegen` or the Inspector (see debugging).

XPath and CSS you still need

WANT TO FIND	CSS SELECTOR	XPATH
By id	`#email`	`//*[@id='email']`
Tag with attribute	`input[type='email']`	`//input[@type='email']`
By exact text	use `getByText`	`//button[text()='Login']`
Part of an attribute	`div[class*='err']`	`//div[contains(@class,'err')]`
Child inside parent	`#product-3 button`	`//div[@id='product-3']//button`
Next to a label	use `getByLabel`	`//label[text()='Email']/following::input[1]`

5. Actions: doing things on an element

Every action first runs **auto-wait checks** (see section 7), so you rarely need extra waits.

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`click()`	Clicks the element. Waits until it is visible, stable, enabled and not covered.	`page.locator("#loginBtn").click();`

IMPORTANT	`fill(String text)`	Clears the input and sets the text in one step. Works on text, date and range inputs.	`page.getByLabel("Email").fill("a@b.com");`
IMPORTANT	`clear()`	Empties an input.	
IMPORTANT	`pressSequentially(String text)`	Types key by key like a real user (slower than fill). Use when the page reacts to each key, or to test `maxlength`.	`box.pressSequentially("hello");`
IMPORTANT	`press(String key)`	Presses a key or key combination.	`box.press("Enter");` `box.press("Control+A");`
IMPORTANT	`check()` / `uncheck()` / `setChecked(boolean)`	Ticks or unticks a checkbox or radio button. Safe: it checks the state first.	`page.getByLabel("English").check();`
IMPORTANT	`selectOption(...)`	Chooses option(s) in a `<select>`. By value or label: `selectOption("India")`; many: `selectOption(new String[]{"A","B"})`; by label: `new SelectOption().setLabel("USA")`; by index: `new SelectOption().setIndex(0)`.	
IMPORTANT	`setInputFiles(...)`	Uploads a file. Use a path `Paths.get("a.pdf")` or an in-memory `new FilePayload(name, mimeType, bytes)`.	
IMPORTANT	`hover()`	Moves the mouse over the element.	`page.locator("#menu").hover();`
GOOD TO KNOW	`dblclick()`	Double click.	

GOOD TO KNOW	`click(new Locator.ClickOptions().setButton(MouseButton.RIGHT))`	Right click. Other options: `setForce(true)`, `setModifiers(...)`, `setPosition(x, y)`, `setTimeout(ms)`.	
GOOD TO KNOW	`focus()` / `blur()`	Move keyboard focus to or away from the element.	
GOOD TO KNOW	`dragTo(Locator target)`	Drag and drop.	`source.dragTo(target);`
GOOD TO KNOW	`scrollIntoViewIfNeeded()`	Scrolls the element into view (click does this automatically).	
GOOD TO KNOW	`dispatchEvent("click")`	Sends a DOM event directly. A last option.	

Reading values from the page

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`innerText()`	The visible text of the element. Returns a String.	`String msg = page.locator("#msg").innerText();`
IMPORTANT	`inputValue()`	What is typed inside an input or textarea.	
IMPORTANT	`getAttribute(String name)`	An attribute value, or null if missing.	`el.getAttribute("data-state")`
IMPORTANT	`isVisible()` / `isHidden()`	True or false right now. No waiting.	
IMPORTANT	`isEnabled()` / `isDisabled()` / `isChecked()` / `isEditable()`	State of the element right now. No waiting.	
GOOD TO KNOW	`textContent()`	All text including hidden parts.	

GOOD TO KNOW	`allInnerTexts()` / `allTextContents()`	Texts of ALL matching elements as `List<String>`.
GOOD TO KNOW	`boundingBox()`	Position and size of the element.

IMPORTANT **is...() vs assertions**

`isVisible()` and the other `is...()` methods give an answer **right now** and do not wait. For tests, prefer the assertions in the next section, which wait and retry.

6. Assertions: checking results

Import this once: `import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;`. Then write `assertThat(locator).hasText("Hello");`. These assertions **retry** for up to 5 seconds until they become true. That is why Playwright tests are stable.

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`assertThat(locator).isVisible()` / `isHidden()`	Element is shown / not shown.	
IMPORTANT	`assertThat(locator).isEnabled()` / `isDisabled()`	Element is clickable / disabled.	
IMPORTANT	`assertThat(locator).isChecked()`	Checkbox or radio is ticked.	
IMPORTANT	`assertThat(locator).hasText("exact text")`	The WHOLE text equals this (spaces are ignored). Pass a `String[]` to check a list of elements, or a `Pattern` for a regular expression.	`assertThat(page.locator("#msg")).hasText("Saved");`

IMPORTANT	`assertThat(locator).containsText("part")`	The text contains this part.	
IMPORTANT	`assertThat(locator).hasValue("abc")`	An input contains this value.	
IMPORTANT	`assertThat(locator).hasAttribute("type", "password")`	An attribute has this value.	
IMPORTANT	`assertThat(locator).hasCount(3)`	Exactly this many elements match.	
IMPORTANT	`assertThat(page).hasURL(...)` / `hasTitle(...)`	Check the address or title. Accepts a String or a Pattern.	
IMPORTANT	`assertThat(locator).not().isChecked()`	`.not()` reverses any check.	
GOOD TO KNOW	`assertThat(locator).isEmpty()` / `isEditable()` / `isFocused()`	Other state checks.	
GOOD TO KNOW	`assertThat(locator).hasClass("active")` / `hasId` / `hasCSS`	Class, id or style checks.	
GOOD TO KNOW	`setDefaultAssertionTimeout(ms)`	Change the 5 second default for all assertions.	`PlaywrightAssertions.setDefaultAssertionTimeout(10000);`

IMPORTANT **Which assert to use**

- Use `assertThat(locator)...` for anything on the page. It waits and retries.

- Use TestNG `Assert.assertEquals(actual, expected)` for plain Java values, such as a number you calculated.
- `hasText("")` with an empty string checks that the element is empty. It does not mean "anything".

---

7. Auto-waiting (the big difference from Selenium)

IMPORTANT **What happens before every click**

Before every action, Playwright waits until the element passes these checks. If they do not pass within 30 seconds, you get a `TimeoutError`.

ACTION	PLAYWRIGHT WAITS UNTIL THE ELEMENT IS
`click()`, `hover()`, `check()`	attached to the page, visible, stable (not moving), enabled, and not covered by another element
`fill()`, `clear()`	visible, enabled and editable
`selectOption()`	visible and enabled
`innerText()`, `getAttribute()`	attached to the page

In Selenium you write `WebDriverWait` and `ExpectedConditions` yourself. In Playwright this is built in. You only need explicit waiting for special cases, such as `locator.waitFor()` or `page.waitForResponse(...)`.

```
page.setDefaultTimeout(10000);                                          // actions wai
page.locator("#late").click(new Locator.ClickOptions().setTimeout(3000));   // this
page.locator("#spinner").waitFor(new Locator.WaitForOptions()
        .setState(WaitForSelectorState.HIDDEN));                        // wait f
```

What it does: Changing the default wait time, one custom wait, and waiting for an element to disappear.

8. Popups, frames, tabs, downloads

IMPORTANT **Alert, confirm, prompt**

Popups: by default Playwright closes (dismisses) alert, confirm and prompt popups by itself. To read one or press OK, register the handler BEFORE the click that opens it.

```
page.onceDialog(dialog → {
    System.out.println(dialog.message());   // read the popup text
    dialog.accept();                        // press OK  (dialog.dismiss() = Ca
});
page.locator("#submitBtn").click();         // this click opens the popup
```

What it does: `onceDialog` handles the next popup only. `onDialog` handles every popup. A handler that neither accepts nor dismisses blocks the page.

```
page.frameLocator("#payFrame").getByLabel("Card number").fill("4111111111111111");
```

What it does: Frames: `frameLocator` goes into an iframe. No switching back is needed afterwards, unlike Selenium.

```
Page newTab = page.waitForPopup(() → page.getByText("Open help").click());
newTab.waitForLoadState();
System.out.println(newTab.title());
```

What it does: New tab: `waitForPopup` runs the click and gives you the new tab as another `Page` object.

```
Download download = page.waitForDownload(() → page.getByText("Export").click());
download.saveAs(Paths.get("report.csv"));
```

What it does: Download: waits for the file and saves it.

```
page.route("**/api/users", route → route.fulfill(
        new Route.FulfillOptions().setStatus(200).setBody("[]")));
```

What it does: Network: replaces the answer of a request with your own. Useful to test an empty list or an error.

```
APIRequestContext request = playwright.request().newContext();
APIResponse response = request.get("https://reqres.in/api/users/2");
System.out.println(response.status());      // 200
System.out.println(response.text());
```

What it does: Playwright can also call APIs directly, without a browser, for API tests or test data setup.

---

9. Setup and cleanup with TestNG

Playwright only drives the browser. We use TestNG (as with Selenium) to run the tests, give annotations and `Assert`. The annotations `@Test`, `@BeforeMethod`, `@AfterMethod`, `@DataProvider` work exactly the same. Here is the base class used in the three cases below.

```java
package playwrighttests;

import com.microsoft.playwright.Browser;
import com.microsoft.playwright.BrowserContext;
import com.microsoft.playwright.BrowserType;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.Playwright;
import org.testng.annotations.AfterClass;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.BeforeMethod;

public class BaseTest {

    protected Playwright playwright;      // the main object: it starts everyt
    protected Browser browser;            // one browser (Chrome), shared by a
    protected BrowserContext context;     // a fresh "profile" for each test (
    protected Page page;                  // one tab inside the context

    @BeforeClass
    public void launchBrowser() {
        playwright = Playwright.create();
        browser = playwright.chromium().launch(new BrowserType.LaunchOptions()
                .setChannel("chrome")                              // use the
                .setHeadless(Boolean.getBoolean("headless")));     // false =
    }

    @BeforeMethod
    public void createPage() {
        context = browser.newContext();       // new context = clean cookies,
        page = context.newPage();
    }

    @AfterMethod(alwaysRun = true)
    public void closeContext() {
        context.close();
    }

    @AfterClass(alwaysRun = true)
    public void closeBrowser() {
        browser.close();
        playwright.close();
    }
}
```

LINES	WHAT THIS PART DOES
1-12	Package and imports from Playwright and TestNG.

13-14	Parent class of the test classes. They `extend` it and get a ready `page`.
15-19	The four objects from the hierarchy: `Playwright`, `Browser`, `BrowserContext`, `Page`.
20-27	Runs once before the tests of a class. Starts Playwright and opens Chrome (one browser is shared, which is fast). `setChannel("chrome")` uses your installed Chrome. `-Dheadless=true` hides the window.
28-33	Runs before every test. A NEW context and page, so every test starts clean with no cookies.
34-38	Closes the context after each test, even if it failed.
39-44	Closes the browser and Playwright at the end of the class.

10. Debugging and tools

	TOOL	WHAT IT DOES	HOW
IMPORTANT	`Run with a visible browser`	Use `setHeadless(false)`, and add `setSlowMo(500)` to watch every step.	`mvn test -Dtest=PasswordTest (without -Dheadless=true)`
IMPORTANT	`page.pause()`	Stops the test and opens the Inspector. You can step through and try locators.	`page.pause();`
IMPORTANT	`Codegen`	Opens a browser, records what you click, and writes the Java code.	`mvn exec:java -e -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="codegen https://example.com"`

GOOD TO KNOW	`Trace viewer`	A recording with a timeline, screenshots, DOM and network of each step. Great for failures in CI.	`context.tracing().start(new Tracing.StartOptions().setScreenshots(true).setSnapshots(true)); ... context.tracing().stop(new Tracing.StopOptions().setPath(Paths.get("trace.zip")));`
GOOD TO KNOW	`Open a trace`	Opens the saved trace in a viewer.	`mvn exec:java -e -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="show-trace trace.zip"`
GOOD TO KNOW	`Video`	Records a video of the test.	`browser.newContext(new Browser.NewContextOptions().setRecordVideoDir(Paths.get("videos/")));`
GOOD TO KNOW	`Debug mode`	Setting the environment variable `PWDEBUG=1` opens the Inspector for the whole run.	`PWDEBUG=1 mvn test -Dtest=PasswordTest`

11. Three real automations

Click a case below to open it. In each case you will see: the practice page, where every element is, the manual test cases, the full Java code with line numbers, and the code explained line by line. **Read the manual test cases first, then the code.** That is how real automation is done: manual thinking first, code second.

IMPORTANT **Before you start**

- Open each practice page in Chrome and test it by hand first. The page links are inside each case below.
- `PAGE_URL` in each test already points to the live practice page on GitHub Pages, so you only need internet. To use your own copy, open the file from `docs/demo-pages/` and put its `file:///` path in `PAGE_URL`.

- Run with Maven: `mvn test -Dtest="playwrighttests.PasswordTest"`. Remove `-Dheadless=true` to watch the browser work.
- All three cases were run and passed (66 checks) on the pages as given.

▼ Case 1: Password validation (a full validation spec)

Boundary values, equivalence partitions, negative and state testing on one password form.

STEP 1: THE PRACTICE PAGE

Open the page, click around like a manual tester first, then look at the locators below.

Open demo-password.html

The test already points to this live page (`PAGE_URL`), so it runs from any computer with internet. The same file is in the repository at `docs/demo-pages/demo-password.html`.

IMPORTANT **What you will learn from this case**

- Data-driven tests (one method runs many times with different data)
- Boundary value analysis and equivalence partitioning in real code
- Checking one rule at a time
- Reading attributes and the enabled / disabled state of a button

STEP 2: WHERE EACH ELEMENT IS (LOCATORS)

ELEMENT	HTML ON THE PAGE	PLAYWRIGHT LOCATOR (BEST)	CSS SELECTOR	XPATH
Username box	`<input id="username" type="text">`	`page.getByLabel("Username")`	`#username`	`//input[@id='username']`
Password box	`<input id="password" type="password">`	`page.getByLabel("Password", exact)`	`input[type='password']`	`//input[@id='password']`

Confirm password box	`<input id="confirmPassword" type="password">`	`page.getByLabel("Confirm password")`	`#confirmPassword`	`//label[text()='Confirm password']/following::input[1]`
Show / Hide button	`<button id="togglePwd">Show</button>`	`page.getByRole(BUTTON, name "Show")`	`#togglePwd`	`//button[text()='Show']`
One rule line	`<li id="rule-length" data-state="pending">`	`page.locator("#rule-length")`	`li[data-state='pass']`	`//li[@id='rule-length']`
Strength text	`<span id="strength">Not entered</span>`	`page.locator("#strength")`	`#strength`	`//span[@id='strength']`
Error message	`<div id="pwdError" class="error">`	`page.locator("#pwdError")`	`#pwdError`	`//div[@id='pwdError']`
Create password button	`<button type="submit" id="submitBtn" disabled>`	`page.getByRole(BUTTON, name "Create password")`	`button[type='submit']`	`//button[@type='submit']`
Success message	`<div id="successMsg" hidden>`	`page.locator("#successMsg")`	`#successMsg`	`//div[contains(text(),'created successfully')]`

STEP 3: MANUAL TEST CASES FIRST

Automation follows manual testing. We first decide what to test, and only then write code.

ID	TECHNIQUE USED	INPUT	EXPECTED RESULT
TC01	State check	Nothing typed	Button disabled, strength "Not entered", rules pending
TC02-05	Positive + boundary	Abcd@1234, Abc@1234 (8 chars), Abcdefgh@1234567 (16 chars), Str0ng@Passw0rd	All rules pass, Medium or Strong, button enabled, success message
TC06-07	Boundary value	7 characters, 17 characters	Only the length rule fails
TC08-11	Equivalence partition	No uppercase, no lowercase, no digit, no special character	Only that one rule fails
TC12	Error guessing	A space inside the password	No-space rule fails
TC13	Business rule	Username ubed, password MyUBED@123	No-username rule fails
TC14-16	Error guessing (hostile input)	SQL text, script text, only spaces	Rejected, button disabled, no popup
TC17-19	Negative: confirm password	Different, different only in case, empty	Error message, button disabled
TC20-21	State transition	Valid then invalid then valid; username changed later	Button enabled or disabled to match the data
TC22	UI behaviour	Click Show / Hide	Field type switches between password and text

STEP 4: THE CODE

File: `playwrighttests/PasswordTest.java`

```java
package playwrighttests;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.options.AriaRole;
import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import java.util.concurrent.atomic.AtomicBoolean;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.ass

public class PasswordTest extends BaseTest {

    // Live practice page hosted on GitHub Pages (works from any computer
    private static final String PAGE_URL = "https://ubedzz49.github.io/qa-

    private static final String[] ALL_RULES =
            {"length", "upper", "lower", "number", "special", "nospace", "

    // ---------- Helper methods ----------
    private Locator passwordBox() {
        return page.getByLabel("Password", new Page.GetByLabelOptions().se
    }

    private void enter(String pwd, String confirm, String user) {
        page.getByLabel("Username").fill(user);
        passwordBox().fill(pwd);
        page.getByLabel("Confirm password").fill(confirm);
    }

    private void enter(String pwd) {
        enter(pwd, pwd, "");
    }

    private void expectRule(String rule, String state) {
        assertThat(page.locator("#rule-" + rule)).hasAttribute("data-state
    }

    private void open() {
        page.navigate(PAGE_URL);
    }

    // ---------- 1. Initial state ----------
    @Test
    public void tc01_initialState() {
        open();
        assertThat(page.locator("#submitBtn")).isDisabled();
```

```
50          assertThat(page.locator("#strength")).hasText("Not entered");
51          for (String rule : ALL_RULES) {
52              expectRule(rule, "pending");
53          }
54          assertThat(page.locator("#pwdError")).isEmpty();
55          assertThat(page.locator("#successMsg")).isHidden();
56      }
57
58      // ---------- 2. Positive cases ----------
59      @DataProvider(name = "positive")
60      public Object[][] positive() {
61          return new Object[][]{
62                  // id,     technique,                       password,
63                  {"TC02", "Valid typical password",        "Abcd@1234",
64                  {"TC03", "Boundary: 8 chars (min)",       "Abc@1234",
65                  {"TC04", "Boundary: 16 chars (max)",      "Abcdefgh@123456
66                  {"TC05", "Strong password (12+ chars)",   "Str0ng@Passw0rd
67          };
68      }
69
70      @Test(dataProvider = "positive")
71      public void positiveCases(String id, String technique, String pwd, Str
72          open();
73          enter(pwd);
74
75          for (String rule : ALL_RULES) {
76              expectRule(rule, "pass");
77          }
78          assertThat(page.locator("#strength")).hasText(expectedStrength);
79          assertThat(page.locator("#pwdError")).isEmpty();
80          assertThat(page.locator("#submitBtn")).isEnabled();
81
82          page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setNam
83          assertThat(page.locator("#successMsg")).hasText("Password created
84      }
85
86      // ---------- 3. Negative cases: break exactly ONE rule at a time ----
87      @DataProvider(name = "negative")
88      public Object[][] negative() {
89          return new Object[][]{
90                  // id,     technique,                         passwor
91                  {"TC06", "Boundary: 7 chars (min - 1)",         "Abc@123"
92                  {"TC07", "Boundary: 17 chars (max + 1)",        "Abcdefgh
93                  {"TC08", "Partition: no uppercase letter",      "abcd@123
94                  {"TC09", "Partition: no lowercase letter",      "ABCD@123
95                  {"TC10", "Partition: no number",                "Abcd@efg
96                  {"TC11", "Partition: no special character",     "Abcd1234
97                  {"TC12", "Error guessing: space inside",        "Abcd @12
98                  {"TC13", "Business rule: contains username",    "MyUBED@1
```

```
        };
    }

    @Test(dataProvider = "negative")
    public void negativeCases(String id, String technique, String pwd, Str
        open();
        enter(pwd, pwd, user);

        for (String rule : ALL_RULES) {
            expectRule(rule, rule.equals(broken) ? "fail" : "pass");
        }
        assertThat(page.locator("#strength")).hasText("Weak");
        assertThat(page.locator("#pwdError")).hasText("Password does not m
        assertThat(page.locator("#submitBtn")).isDisabled();
    }

    // ---------- 4. Error guessing: hostile and strange input ----------
    @Test
    public void tc14_sqlInjectionTextIsRejected() {
        open();
        enter("' OR '1'='1");
        assertThat(page.locator("#submitBtn")).isDisabled();
        assertThat(page.locator("#successMsg")).isHidden();
    }

    @Test
    public void tc15_scriptTextShowsNoPopup() {
        open();
        AtomicBoolean popupSeen = new AtomicBoolean(false);
        page.onDialog(dialog -> {            // runs only if a popup appea
            popupSeen.set(true);
            dialog.dismiss();
        });
        enter("<script>alert(1)</script>");
        assertThat(page.locator("#submitBtn")).isDisabled();
        Assert.assertFalse(popupSeen.get(), "An unexpected popup appeared"
    }

    @Test
    public void tc16_onlySpacesAreRejected() {
        open();
        enter("        ");
        expectRule("nospace", "fail");
        assertThat(page.locator("#submitBtn")).isDisabled();
    }

    // ---------- 5. Confirm password ----------
    @Test
    public void tc17_mismatchShowsError() {
```

```
148         open();
149         enter("Abcd@1234", "Abcd@12345", "");
150         assertThat(page.locator("#confirmError")).hasText("Passwords do no
151         assertThat(page.locator("#submitBtn")).isDisabled();
152     }
153 
154     @Test
155     public void tc18_caseDifferenceIsMismatch() {
156         open();
157         enter("Abcd@1234", "abcd@1234", "");
158         assertThat(page.locator("#confirmError")).hasText("Passwords do no
159     }
160 
161     @Test
162     public void tc19_emptyConfirmKeepsButtonDisabled() {
163         open();
164         enter("Abcd@1234", "", "");
165         assertThat(page.locator("#submitBtn")).isDisabled();
166     }
167 
168     // ---------- 6. State transitions ----------
169     @Test
170     public void tc20_validInvalidValidChangesButton() {
171         open();
172         Locator button = page.locator("#submitBtn");
173         enter("Abcd@1234");
174         assertThat(button).isEnabled();
175 
176         passwordBox().fill("abcd@1234");                    // breaks th
177         assertThat(button).isDisabled();
178 
179         passwordBox().fill("Abcd@1234");
180         assertThat(button).isEnabled();
181     }
182 
183     @Test
184     public void tc21_changingUsernameCanBreakValidPassword() {
185         open();
186         enter("Abcd@1234");
187         assertThat(page.locator("#submitBtn")).isEnabled();
188         page.getByLabel("Username").fill("abcd");
189         expectRule("nouser", "fail");
190         assertThat(page.locator("#submitBtn")).isDisabled();
191     }
192 
193     // ---------- 7. UI behaviour ----------
194     @Test
195     public void tc22_showHideTogglesFieldType() {
196         open();
```

```
        Locator toggle = page.locator("#togglePwd");
        assertThat(passwordBox()).hasAttribute("type", "password");
        toggle.click();
        assertThat(passwordBox()).hasAttribute("type", "text");
        assertThat(toggle).hasText("Hide");
        toggle.click();
        assertThat(passwordBox()).hasAttribute("type", "password");
    }
}
```

STEP 5: EXPLAINED LINE BY LINE

LINES	WHAT THIS PART DOES
1-13	Package and imports. `assertThat` is the retrying Playwright assertion. `AtomicBoolean` holds a flag that a popup listener can change.
14-16	Extends `BaseTest`, so `page` is ready. Every `@Test` method gets a fresh context and page.
17-18	Address of the practice page. Change `PATH/TO` to your folder.
19-21	The seven rule names. We loop over them instead of writing seven lines each time.
22	Small helper methods, so the tests below read clearly.
23-26	`getByLabel("Password")` would also match "Confirm password". `setExact(true)` makes it match only the label "Password".
27-36	`enter()` fills three boxes with `fill()`. `getByLabel` finds an input by its label, like a user would. The short `enter(pwd)` below is an overloaded shortcut.
37-44	Checks the `data-state` attribute of a rule line. The page sets it to `pass`, `fail` or `pending`. `assertThat(...).hasAttribute` retries until true.
45-57	TC01: before typing anything the button is disabled, strength says "Not entered", every rule is pending, and there is no error or success text.
58-69	Data provider with four valid passwords. Two are the boundaries 8 and 16 characters. One test method runs four times.
70-85	Types the password, checks all rules pass, the strength text, no error and an enabled button. Then it finds the button by its role and name, clicks it, and checks the success message.

86-101	Eight invalid passwords. Each breaks exactly ONE rule: boundary 7 and 17, no uppercase / lowercase / number / special, a space, and the username rule.
102-114	For each rule it expects `fail` only for the broken rule and `pass` for all the others. This proves the page flags the right rule and nothing else. It also checks "Weak", the error text, and a disabled button.
115-144	TC14: SQL text. TC15: script text; `onDialog` would set the flag if a popup ever appeared, and we assert that it did not. TC16: only spaces.
145-167	Different confirm password, a difference only in letter case, and an empty confirm password.
168-192	The button must follow the data: valid, then invalid, then valid again. Changing the username later can also break a valid password.
193-205	The Show / Hide button changes the field `type` between `password` and `text`, and its own text between Show and Hide.

STEP 6: RUN IT

```
mvn test -Dtest="playwrighttests.PasswordTest"                  // run this clas
mvn test -Dtest="playwrighttests.PasswordTest" -Dheadless=true   // run without
mvn test                                                       // run every test
```

▼ Case 2: Full page validation (every kind of input)

Text boxes, date, radio, checkbox, dropdowns, multi-select, slider, textarea, file upload and popups.

STEP 1: THE PRACTICE PAGE

Open the page, click around like a manual tester first, then look at the locators below.

Open demo-form.html

The test already points to this live page (`PAGE_URL`), so it runs from any computer with internet. The same file is in the repository at `docs/demo-pages/demo-form.html`.

IMPORTANT **What you will learn from this case**

- Every input type and how to handle it
- Dependent dropdowns and multi-select

- Alert and confirm popups
- File upload and boundary testing on file size

STEP 2: WHERE EACH ELEMENT IS (LOCATORS)

ELEMENT	HTML ON THE PAGE	PLAYWRIGHT LOCATOR (BEST)	CSS SELECTOR	XPATH
Full name box	`<input id="fullName" type="text">`	`page.getByLabel("Full name")`	`#fullName`	`//input[@id='fullName']`
Date of birth	`<input id="dob" type="date">`	`page.getByLabel("Date of birth")`	`input[type='date']`	`//input[@id='dob']`
Male radio button	`<input type="radio" name="gender" id="genderMale" value="Male">`	`page.getByLabel("Male", exact)`	`input[name='gender'][value='Male']`	`//input[@name='gender' and @value='Male']`
Country dropdown	`<select id="country">`	`page.locator("#country")`	`select#country`	`//select[@id='country']`
State dropdown (starts disabled)	`<select id="state" disabled>`	`page.locator("#state")`	`select#state`	`//select[@id='state']`
Skills multi-select	`<select id="skills" multiple>`	`page.locator("#skills")`	`select[multiple]`	`//select[@multiple]`
English checkbox	`<input type="checkbox" name="lang" id="langEnglish" value="English">`	`page.getByLabel("English")`	`input[name='lang']:checked`	`//input[@name='lang' and @value='English']`

Experience slider	`<input type="range" id="experience" min="0" max="20">`	`page.locator("#experience")`	`input[type='range']`	`//input[@type='range']`
Address textarea	`<textarea id="address" maxlength="200">`	`page.getByLabel("Address")`	`textarea#address`	`//textarea[@id='address']`
Resume file input	`<input type="file" id="resume">`	`page.getByLabel(/Resume/)`	`input[type='file']`	`//input[@type='file']`
Error under a field	`<div class="error" id="err-email">`	`page.locator("#err-email")`	`#err-email`	`//div[@id='err-email']`
Submit / Reset buttons	`<button type="submit" id="submitBtn">Submit</button>`	`page.getByRole(BUTTON, name "Submit")`	`button[type='submit']`	`//button[text()='Reset']`

STEP 3: MANUAL TEST CASES FIRST

Automation follows manual testing. We first decide what to test, and only then write code.

ID	CONTROL	WHAT IS TESTED	TECHNIQUE
TC01-02	Whole form	Start state; submit empty form shows 11 error messages	State check, required fields
TC03	Name, email, phone (text)	Length 1, 2, 40, 41; letters only; email formats; phone length and first digit	Boundary value, equivalence partition
TC04	Date of birth (date)	Exactly 18 years, one day short, one day over, future date	Boundary value
TC05	Gender (radio)	Only one can be selected at a time	State check

TC06-07	Country and State (dependent dropdowns)	State list changes with country; disabled again when country is cleared	Dependency check
TC08	Skills (multi-select)	1 to 3 allowed; 4 and 0 give errors	Boundary value
TC09	Languages (checkboxes)	Tick and untick, count of ticked boxes	State check
TC10	Experience (slider)	Value display at 12 and at maximum 20	Boundary value
TC11-12	Address (textarea)	Counter, 200 character limit, minimum 10 characters	Boundary value
TC13	Resume (file upload)	PDF, PNG, exactly 2 MB, 2 MB + 1 byte, empty file, cv.pdf.exe	Boundary value, error guessing
TC14	Terms (checkbox)	Must be ticked	Required field
TC15	Submit	Full valid form, alert popup, result text	Positive end-to-end
TC16	Reset	Confirm popup: Cancel keeps data, OK clears it	Popup handling

STEP 4: THE CODE

File: `playwrighttests/FormTest.java`

```java
package playwrighttests;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.options.AriaRole;
import com.microsoft.playwright.options.FilePayload;
import com.microsoft.playwright.options.SelectOption;
import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import java.time.LocalDate;
import java.util.concurrent.atomic.AtomicReference;
import java.util.regex.Pattern;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.ass

public class FormTest extends BaseTest {

    // Live practice page hosted on GitHub Pages (works from any computer
    private static final String PAGE_URL = "https://ubedzz49.github.io/qa-
    private static final int MB = 1024 * 1024;

    // ---------- Helper methods ----------
    private void open() {
        page.navigate(PAGE_URL);
    }

    private void clickSubmit() {
        page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setNam
    }

    private Locator error(String field) {
        return page.locator("#err-" + field);
    }

    // ---------- 1. Initial state ----------
    @Test
    public void tc01_initialState() {
        open();
        assertThat(page.locator("#state")).isDisabled();
        assertThat(page.locator("#expOut")).hasText("0");
        assertThat(page.locator("#addrCount")).hasText("0");
        assertThat(page.locator("input[name='gender']:checked")).hasCount(
        assertThat(page.locator("#terms")).not().isChecked();
        assertThat(page.locator("#result")).isHidden();
    }

    // ---------- 2. Empty form: every required field shows an error -----
```

```
50      @Test
51      public void tc02_emptyFormShowsAllErrors() {
52          open();
53          clickSubmit();
54          String[][] expected = {
55                  {"fullName", "Full name is required"},
56                  {"email", "Email is required"},
57                  {"phone", "Mobile number is required"},
58                  {"dob", "Date of birth is required"},
59                  {"gender", "Select a gender"},
60                  {"country", "Select a country"},
61                  {"skills", "Select at least one skill"},
62                  {"lang", "Select at least one language"},
63                  {"address", "Address is required"},
64                  {"resume", "Upload your resume"},
65                  {"terms", "You must accept the terms"},
66          };
67          for (String[] row : expected) {
68              assertThat(error(row[0])).hasText(row[1]);
69          }
70      }
71
72      // ---------- 3. Text inputs: data-driven validation ----------
73      @DataProvider(name = "textCases")
74      public Object[][] textCases() {
75          String phoneMsg = "Enter a 10 digit number starting with 6, 7, 8 o
76          String nameLenMsg = "Name must be 2 to 40 characters";
77          return new Object[][]{
78                  // field,      value,              expected message,  why
79                  {"fullName", "A",                nameLenMsg, "Boundary: 1
80                  {"fullName", "Al",               "",         "Boundary: 2
81                  {"fullName", "A".repeat(40),     "",         "Boundary: 4
82                  {"fullName", "A".repeat(41),     nameLenMsg, "Boundary: 4
83                  {"fullName", "John3",            "Name can have only lett
84                  {"fullName", "Mary-Ann O'Neil",  "",         "Valid speci
85                  {"email",    "abc",              "Enter a valid email add
86                  {"email",    "abc@mail",         "Enter a valid email add
87                  {"email",    "a b@mail.com",     "Enter a valid email add
88                  {"email",    "user@mail.com",    "",         "Valid email
89                  {"phone",    "12345",            phoneMsg,   "Invalid: to
90                  {"phone",    "5876543210",       phoneMsg,   "Invalid: st
91                  {"phone",    "98765432101",      phoneMsg,   "Boundary: 1
92                  {"phone",    "abcdefghij",       phoneMsg,   "Invalid: le
93                  {"phone",    "9876543210",       "",         "Valid mobil
94          };
95      }
96
97      @Test(dataProvider = "textCases")
98      public void tc03_textFieldValidation(String field, String value, Strin
```

```
99          open();
100         page.locator("#" + field).fill(value);
101         clickSubmit();
102         assertThat(error(field)).hasText(message);
103     }
104
105     // ---------- 4. Date of birth: boundary around 18 years ----------
106     @DataProvider(name = "dobCases")
107     public Object[][] dobCases() {
108         LocalDate today = LocalDate.now();
109         return new Object[][]{
110                 {"exactly 18 years today (valid)",  today.minusYears(18).t
111                 {"turns 18 tomorrow (invalid)",     today.minusYears(18).p
112                 {"18 years and 1 day (valid)",      today.minusYears(18).m
113                 {"tomorrow (future date)",          today.plusDays(1).toSt
114         };
115     }
116
117     @Test(dataProvider = "dobCases")
118     public void tc04_dateOfBirth(String why, String date, String message)
119         open();
120         page.locator("#dob").fill(date);                    // fill() work
121         clickSubmit();
122         assertThat(error("dob")).hasText(message);
123     }
124
125     // ---------- 5. Radio buttons ----------
126     @Test
127     public void tc05_onlyOneRadioCanBeSelected() {
128         open();
129         page.getByLabel("Female").check();
130         assertThat(page.getByLabel("Female")).isChecked();
131         page.getByLabel("Other").check();
132         assertThat(page.getByLabel("Other")).isChecked();
133         assertThat(page.getByLabel("Female")).not().isChecked();
134         assertThat(page.locator("input[name='gender']:checked")).hasCount(
135     }
136
137     // ---------- 6. Dropdowns: single and dependent ----------
138     @Test
139     public void tc06_countryChangesStateList() {
140         open();
141         Locator state = page.locator("#state");
142
143         page.locator("#country").selectOption("India");
144         assertThat(state).isEnabled();
145         assertThat(state.locator("option")).hasText(
146                 new String[]{"-- Select state --", "Karnataka", "Maharasht
147
```

```
148         page.locator("#country").selectOption(new SelectOption().setLabel(
149         assertThat(state.locator("option")).hasText(
150                 new String[]{"-- Select state --", "California", "Texas"})
151 
152         page.locator("#country").selectOption(new SelectOption().setIndex(
153         assertThat(state).isDisabled();
154     }
155 
156     @Test
157     public void tc07_stateIsRequiredAfterChoosingCountry() {
158         open();
159         page.locator("#country").selectOption("UK");
160         clickSubmit();
161         assertThat(error("state")).hasText("Select a state");
162     }
163 
164     // ---------- 7. Multi-select ----------
165     @Test
166     public void tc08_multiSelectAllowsOneToThreeSkills() {
167         open();
168         Locator skills = page.locator("#skills");
169 
170         skills.selectOption(new String[]{"Selenium", "Git"});
171         clickSubmit();
172         assertThat(error("skills")).hasText("");
173 
174         skills.selectOption(new String[]{"Selenium", "Playwright", "SQL",
175         clickSubmit();
176         assertThat(error("skills")).hasText("Select at most 3 skills");
177 
178         skills.selectOption(new String[]{});
179         clickSubmit();
180         assertThat(error("skills")).hasText("Select at least one skill");
181     }
182 
183     // ---------- 8. Checkboxes ----------
184     @Test
185     public void tc09_languageCheckboxes() {
186         open();
187         page.getByLabel("English").check();
188         page.getByLabel("Telugu").check();
189         assertThat(page.locator("input[name='lang']:checked")).hasCount(2)
190         page.getByLabel("English").uncheck();
191         assertThat(page.locator("input[name='lang']:checked")).hasCount(1)
192     }
193 
194     // ---------- 9. Slider ----------
195     @Test
196     public void tc10_sliderShowsItsValue() {
```

```
197         open();
198         page.locator("#experience").fill("12");
199         assertThat(page.locator("#expOut")).hasText("12");
200         page.locator("#experience").fill("20");                // max
201         assertThat(page.locator("#expOut")).hasText("20");
202     }
203 
204     // ---------- 10. Textarea ----------
205     @Test
206     public void tc11_textareaCounterAndLimit() {
207         open();
208         Locator box = page.locator("#address");
209         box.pressSequentially("Hello");
210         assertThat(page.locator("#addrCount")).hasText("5");
211 
212         box.fill("");
213         box.pressSequentially("x".repeat(205));                // we type 2
214         assertThat(page.locator("#addrCount")).hasText("200");
215     }
216 
217     @Test
218     public void tc12_shortAddressIsRejected() {
219         open();
220         page.locator("#address").fill("Short");
221         clickSubmit();
222         assertThat(error("address")).hasText("Address must be at least 10
223     }
224 
225     // ---------- 11. File upload ----------
226     @DataProvider(name = "fileCases")
227     public Object[][] fileCases() {
228         return new Object[][]{
229                 // why,                       file name,      size,
230                 {"valid PDF",               "resume.pdf",  1000,        ""
231                 {"wrong type (png)",        "photo.png",   1000,        "O
232                 {"exactly 2 MB (max)",      "big.pdf",     2 * MB,      ""
233                 {"2 MB + 1 byte (max + 1)", "bigger.pdf",  2 * MB + 1,  "F
234                 {"empty file",              "empty.pdf",   0,           "F
235                 {"double extension",        "cv.pdf.exe",  1000,        "O
236         };
237     }
238 
239     @Test(dataProvider = "fileCases")
240     public void tc13_resumeUpload(String why, String name, int size, Strin
241         open();
242         // Playwright can upload a file made in memory. No real file is ne
243         page.locator("#resume").setInputFiles(new FilePayload(name, "appli
244         clickSubmit();
245         assertThat(error("resume")).hasText(message);
```

```
246     }
247
248     // ---------- 12. Terms checkbox ----------
249     @Test
250     public void tc14_termsMustBeAccepted() {
251         open();
252         clickSubmit();
253         assertThat(error("terms")).hasText("You must accept the terms");
254         page.locator("#terms").check();
255         clickSubmit();
256         assertThat(error("terms")).hasText("");
257     }
258
259     // ---------- 13. Happy path: whole form, plus the alert popup -------
260     @Test
261     public void tc15_validFormSubmitsAndShowsResult() {
262         open();
263         page.getByLabel("Full name").fill("Ravi Kumar");
264         page.getByLabel("Email").fill("ravi@example.com");
265         page.getByLabel("Mobile number").fill("9876543210");
266         page.getByLabel("Date of birth").fill("1998-05-20");
267         page.getByLabel("Male", new Page.GetByLabelOptions().setExact(true
268         page.getByLabel("Country").selectOption("India");
269         page.getByLabel("State").selectOption("Karnataka");
270         page.getByLabel(Pattern.compile("Skills")).selectOption(new String
271         page.getByLabel("English").check();
272         page.getByLabel("Hindi").check();
273         page.getByLabel(Pattern.compile("Experience in years")).fill("2");
274         page.getByLabel("Address").fill("12 MG Road, Bengaluru");
275         page.getByLabel(Pattern.compile("Resume")).setInputFiles(
276                 new FilePayload("ravi.pdf", "application/pdf", "dummy pdf"
277         page.getByLabel("I agree to the terms and conditions").check();
278
279         // The page shows an alert popup after a good submit. Listen BEFOR
280         AtomicReference<String> popupText = new AtomicReference<>("");
281         page.onceDialog(dialog -> {
282             popupText.set(dialog.message());
283             dialog.accept();
284         });
285         clickSubmit();
286
287         Assert.assertEquals(popupText.get(), "Application submitted succes
288         Locator result = page.locator("#result");
289         assertThat(result).containsText("\"fullName\": \"Ravi Kumar\"");
290         assertThat(result).containsText("\"phone\": \"9876543210\"");
291         assertThat(result).containsText("\"state\": \"Karnataka\"");
292         assertThat(result).containsText("\"experience\": 2");
293         assertThat(result).containsText("\"resume\": \"ravi.pdf\"");
294     }
```

```
295
296     // ---------- 14. Reset button and the confirm popup ----------
297     @Test
298     public void tc16_resetAsksForConfirmation() {
299         open();
300         page.getByLabel("Full name").fill("Ravi Kumar");
301         page.locator("#country").selectOption("USA");
302
303         page.onceDialog(dialog -> dialog.dismiss());                // pr
304         page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setNam
305         assertThat(page.getByLabel("Full name")).hasValue("Ravi Kumar");
306
307         page.onceDialog(dialog -> dialog.accept());                 // pr
308         page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setNam
309         assertThat(page.getByLabel("Full name")).hasValue("");
310         assertThat(page.locator("#state")).isDisabled();
311     }
312 }
```

STEP 5: EXPLAINED LINE BY LINE

LINES	WHAT THIS PART DOES
1-17	Imports. `SelectOption` for dropdowns, `FilePayload` for in-memory file upload, `AtomicReference` to carry a popup message out of a listener.
18-23	Extends `BaseTest`. `PAGE_URL` is the page address. `MB` is the number of bytes in 1 MB, used for the file size boundaries.
24-36	`open()`, `clickSubmit()` (button found by role and name) and `error(field)`, which returns the red message under a field (`err-<field>`).
37-48	TC01: the State dropdown is disabled, counters show 0, no radio is selected, terms are not ticked, the result is hidden.
49-71	TC02: click Submit on an empty form. A 2D array holds each field and its expected message, and a loop checks all 11.
72-96	Data provider with 15 rows: boundaries on name length (1, 2, 40, 41), invalid characters, email formats, phone length and first digit. An empty expected message means "no error expected".
97-104	One method for all 15 rows: fill the field, click Submit, compare the error text.

105-124	Dates come from `LocalDate.now()`, so the test never goes out of date. `fill()` works directly on a date input. Boundary: exactly 18 years, one day too young, one day older, and a future date.
125-136	`check()` on a radio. Selecting another unselects the first. `hasCount(1)` on the `:checked` selector proves only one is selected.
137-163	This is a dependent dropdown. `selectOption("India")` picks by value; `setLabel("USA")` by visible text; `setIndex(0)` by position. `hasText(String[])` checks the whole list of options in order.
164-182	`selectOption(new String[]{...})` selects many options at once. 2 is valid, 4 is max + 1 (error), and an empty array clears all (error).
183-193	`check()` and `uncheck()` are safe because they look at the current state first. A count of `:checked` boxes proves the result.
194-203	`fill("12")` works directly on a range input. The shown number is checked, including the maximum 20.
204-224	`pressSequentially` types key by key. We type 205 characters and the browser keeps only 200 (`maxlength`). `fill()` would ignore that limit, so it is not used here.
225-247	`setInputFiles(new FilePayload(...))` uploads a file made in memory, so no real file is needed. Boundary: exactly 2 MB passes, 2 MB + 1 byte fails. Also a wrong type, an empty file, and a double extension.
248-258	Without ticking the terms an error shows. After `check()` it clears.
259-295	`getByLabel` fills every control. The popup handler is registered BEFORE the click (`onceDialog`), it saves the message and accepts. After the click we check the message with TestNG `Assert`, and the result text with `containsText`.
296-312	Reset opens a confirm popup. `dismiss()` (Cancel) keeps the data. `accept()` (OK) clears the form and disables the State box.

STEP 6: RUN IT

```
mvn test -Dtest="playwrighttests.FormTest"                  // run this class on
mvn test -Dtest="playwrighttests.FormTest" -Dheadless=true    // run without a b
mvn test                                                   // run every test
```

▼ Case 3: Complete flow (login to order confirmation)

One user journey: login, search, cart, coupon, checkout, payment and logout.

STEP 1: THE PRACTICE PAGE

Open the page, click around like a manual tester first, then look at the locators below.

Open demo-shop.html

The test already points to this live page (`PAGE_URL`), so it runs from any computer with internet. The same file is in the repository at `docs/demo-pages/demo-shop.html` .

IMPORTANT **What you will learn from this case**

- An end-to-end test with many steps
- Checking calculations (subtotal, discount, total)
- Negative cases inside a flow
- Finding elements inside a card or table row

STEP 2: WHERE EACH ELEMENT IS (LOCATORS)

ELEMENT	HTML ON THE PAGE	PLAYWRIGHT LOCATOR (BEST)	CSS SELECTOR	XPATH
Login email box	`<input id="email" type="email">`	`page.getByLabel("Email")`	`#email`	`//input[@id='email']`
Login button	`<button type="button" id="loginBtn">Login</button>`	`page.getByRole(BUTTON, name "Login")`	`#loginBtn`	`//button[text()='Login']`
Search box	`<input id="search" placeholder="Search products">`	`page.getByPlaceholder("Search products")`	`input[placeholder='Search products']`	`//input[@placeholder='Search products']`
A product card	`<div class="product" id="product-2">`	`page.locator("#product-2")`	`.product`	`//div[@id='product-2']`

Add button inside a card	`<button id="add-3">Add to cart</button>`	`page.locator("#product-3").getByRole(BUTTON)`	`#product-3 button`	`//div[@id='product-3']//button`
Cart link	`<button id="cartLink">Cart (<span id="cartCount">0</span>) </button>`	`page.getByRole(BUTTON, name /Cart/)`	`#cartLink`	`//button[contains(text(),'Cart')]`
Quantity of an item	`<span id="qty-1">2</span>`	`page.locator("#qty-1")`	`#cart-row-1 span`	`//tr[@id='cart-row-1']//span`
Remove button	`<button id="rm-3">Remove</button>`	`page.locator("#rm-3")`	`#rm-3`	`//tr[@id='cart-row-3']//button[text()='Remove']`
Coupon box	`<input id="couponInput" aria-label="Coupon code">`	`page.getByLabel("Coupon code")`	`#couponInput`	`//input[@id='couponInput']`
Total amount	`<strong id="total">`	`page.locator("#total")`	`#total`	`//strong[@id='total']`
UPI radio button	`<input type="radio" name="pay" id="payUpi" value="UPI">`	`page.getByLabel("UPI")`	`input[name='pay'] [value='UPI']`	`//input[@name='pay' and @value='UPI']`
Order id	`<strong id="orderId">`	`page.locator("#orderId")`	`#orderId`	`//strong[@id='orderId']`

STEP 3: MANUAL TEST CASES FIRST

Automation follows manual testing. We first decide what to test, and only then write code.

STEP	ACTION	EXPECTED RESULT	IDEA
Login (4 cases)	Empty fields, wrong password, unknown email, password in wrong case	Correct error message, shop page stays hidden	Negative testing
1	Login with qa@test.com and Test@1234	Welcome text, 3 products shown	Positive
2	Search "pen", "zzz"; filter by category	1 product, "No products found", counts correct	Search and filter
3	Add Notebook twice and Backpack once	Cart count is 3	State change
4	Open the cart	Notebook qty 2, line total 500, subtotal 1700	Calculation check
5	Click + then -	Quantity 3 then back to 2	State change
6	Coupon ABC, then SAVE10	Invalid message; then discount 170, total 1530	Negative then positive
7	Remove Backpack	Subtotal 500, discount 50, total 450	Recalculation
8	Checkout with empty fields, PIN 123, no payment	Three different error messages	Validation
9	Pay with UPI	Order placed, id like ORD-12345, total 450, cart emptied	End result
10	Logout	Back to login page	Flow end
Extra	Open cart with no items	Checkout button disabled	Rule check

STEP 4: THE CODE

File: `playwrighttests/ShopFlowTest.java`

```java
package playwrighttests;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.options.AriaRole;
import com.microsoft.playwright.options.SelectOption;
import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import java.util.regex.Pattern;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.ass

public class ShopFlowTest extends BaseTest {

    // Live practice page hosted on GitHub Pages (works from any computer
    private static final String PAGE_URL = "https://ubedzz49.github.io/qa-
    private static final String EMAIL = "qa@test.com";
    private static final String PASSWORD = "Test@1234";

    // ---------- Helper methods ----------
    private void login(String email, String password) {
        page.getByLabel("Email").fill(email);
        page.getByLabel("Password").fill(password);
        page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setNam
    }

    // "₹1530" → 1530
    private int money(String css) {
        return Integer.parseInt(page.locator(css).innerText().replace("₹",
    }

    // ---------- Login: negative cases ----------
    @DataProvider(name = "badLogins")
    public Object[][] badLogins() {
        return new Object[][]{
                {"both empty",             "",                "",
                {"wrong password",         EMAIL,             "wrong",
                {"unknown email",          "nobody@test.com", PASSWORD,
                {"password in wrong case", EMAIL,             "test@1234"
        };
    }

    @Test(dataProvider = "badLogins")
    public void loginFails(String why, String email, String password, Stri
        page.navigate(PAGE_URL);
        login(email, password);
        assertThat(page.locator("#loginError")).hasText(message);
```

```
        assertThat(page.locator("#shopView")).isHidden();
    }

    // ---------- Complete end-to-end flow ----------
    @Test
    public void endToEndPurchase() {
        page.navigate(PAGE_URL);

        // 1. Login with valid details
        login(EMAIL, PASSWORD);
        assertThat(page.locator("#welcome")).hasText("Welcome, qa@test.com
        assertThat(page.locator(".product")).hasCount(3);

        // 2. Search and filter products
        Locator search = page.getByPlaceholder("Search products");
        search.fill("pen");
        assertThat(page.locator(".product")).hasCount(1);
        assertThat(page.locator("#product-2 .name")).hasText("Pen Pack");

        search.fill("zzz");
        assertThat(page.locator("#noResults")).isVisible();

        search.fill("");
        page.getByLabel("Category").selectOption("Bags");
        assertThat(page.locator(".product")).hasCount(1);
        page.getByLabel("Category").selectOption("All");
        assertThat(page.locator(".product")).hasCount(3);

        // 3. Add products to the cart
        page.locator("#add-1").click();                  // Notebook
        page.locator("#add-1").click();                  // Notebook again
        page.locator("#product-3").getByRole(AriaRole.BUTTON).click();
        assertThat(page.locator("#cartCount")).hasText("3");

        // 4. Check the cart and the maths
        page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setNam
        assertThat(page.locator("#qty-1")).hasText("2");
        assertThat(page.locator("#cart-row-1 .line-total")).hasText("₹500"
        Assert.assertEquals(money("#subtotal"), 500 + 1200);
        Assert.assertEquals(money("#discount"), 0);
        Assert.assertEquals(money("#total"), 1700);

        // 5. Change quantity
        page.locator("#inc-1").click();
        assertThat(page.locator("#qty-1")).hasText("3");
        page.locator("#dec-1").click();
        assertThat(page.locator("#qty-1")).hasText("2");

        // 6. Coupon: invalid first, then valid
```

```
        page.getByLabel("Coupon code").fill("ABC");
        page.locator("#applyCoupon").click();
        assertThat(page.locator("#couponMsg")).hasText("Invalid coupon cod
        Assert.assertEquals(money("#discount"), 0);

        page.getByLabel("Coupon code").fill("SAVE10");
        page.locator("#applyCoupon").click();
        assertThat(page.locator("#couponMsg")).hasText("Coupon SAVE10 appl
        Assert.assertEquals(money("#discount"), 170);
        Assert.assertEquals(money("#total"), 1530);

        // 7. Remove an item and the total updates
        page.locator("#rm-3").click();                    // remove Backpac
        assertThat(page.locator("#cart-row-3")).hasCount(0);
        Assert.assertEquals(money("#subtotal"), 500);
        Assert.assertEquals(money("#discount"), 50);
        Assert.assertEquals(money("#total"), 450);

        // 8. Checkout: validation first
        page.locator("#checkoutBtn").click();
        page.locator("#placeOrder").click();
        assertThat(page.locator("#checkoutError")).hasText("Name and addre

        page.getByLabel("Full name").fill("Ravi Kumar");
        page.getByLabel("Address").fill("12 MG Road, Bengaluru");
        page.getByLabel("PIN code").fill("123");
        page.locator("#placeOrder").click();
        assertThat(page.locator("#checkoutError")).hasText("Enter a valid

        page.getByLabel("PIN code").fill("560001");
        page.locator("#placeOrder").click();
        assertThat(page.locator("#checkoutError")).hasText("Select a payme

        // 9. Pay and confirm the order
        page.getByLabel("UPI").check();
        page.locator("#placeOrder").click();
        assertThat(page.locator("#orderStatus")).hasText("Order placed");
        assertThat(page.locator("#orderId")).hasText(Pattern.compile("^ORD
        assertThat(page.locator("#orderTotal")).hasText("₹450");
        assertThat(page.locator("#orderPay")).hasText("UPI");
        assertThat(page.locator("#cartCount")).hasText("0");          // c

        // 10. Logout returns to the login page
        page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setNam
        assertThat(page.locator("#loginView")).isVisible();
        assertThat(page.locator("#topNav")).isHidden();
    }

    // ---------- Empty cart rule ----------
```

```
148     @Test
149     public void checkoutIsDisabledForEmptyCart() {
150         page.navigate(PAGE_URL);
151         login(EMAIL, PASSWORD);
152         page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setNam
153         assertThat(page.locator("#emptyCart")).isVisible();
154         assertThat(page.locator("#checkoutBtn")).isDisabled();
155     }
156 }
```

STEP 5: EXPLAINED LINE BY LINE

LINES	WHAT THIS PART DOES
1-14	Imports. `Pattern` is used for regular expressions.
15-21	Extends `BaseTest`. The page address and the valid login are constants.
22-29	`login()` fills the two boxes using labels and clicks the Login button found by role and name.
30-33	`money("#total")` turns the text "₹1530" into the number 1530 so we can compare numbers with TestNG `Assert`.
34-44	Four wrong logins: both empty, wrong password, unknown email, and the password in the wrong letter case.
45-52	Each row logs in with bad data, then checks the exact error text and that the shop page stays hidden.
53-57	One long test is one full user journey. The numbered comments are the steps.
58-62	Log in and check the welcome text and 3 products. No explicit wait is needed, `assertThat` retries.
63-77	`getByPlaceholder` finds the search box. "pen" shows 1 product; "zzz" shows "No products found". `selectOption` filters by category.
78-83	Add the Notebook twice. The Backpack button is found INSIDE its card: `locator("#product-3").getByRole(BUTTON)` (chaining).
84-91	Open the cart (a regular expression matches "Cart (3)"), check quantity 2, the line total, and that subtotal 1700 = 500 + 1200.
92-97	Plus makes quantity 3, minus brings it back to 2.

98-109	A wrong coupon gives an error and no discount. SAVE10 gives 10% off: discount 170, total 1530.
110-116	Remove the Backpack. `hasCount(0)` proves the row is gone, and the totals recalculate: 500, 50, 450.
117-131	Three validation messages in order: empty fields, wrong PIN 123, no payment method.
132-140	Choose UPI and place the order. Check the status, the order id with a regular expression (`ORD-` and 5 digits), the amount, and that the cart is empty.
141-146	Logout returns to the login page and hides the top menu.
147-156	A separate small test: with an empty cart the Checkout button must be disabled.

STEP 6: RUN IT

```
mvn test -Dtest="playwrighttests.ShopFlowTest"                  // run this clas
mvn test -Dtest="playwrighttests.ShopFlowTest" -Dheadless=true   // run without
mvn test                                                       // run every test
```

12. Selenium vs Playwright (a favourite interview question)

	SELENIUM	PLAYWRIGHT
Waiting	You write explicit waits yourself	**Auto-wait** built in
Assertions	TestNG or JUnit, no retry	`assertThat(locator)` retries until true
Speed	Slower, goes through a driver per browser	Faster, one connection per browser
Isolation	New driver per test (slow)	New context per test (fast, cheap)
Browsers	Chrome, Firefox, Edge, Safari	Chromium, Firefox, WebKit
Languages	Java, Python, C#, JS, Ruby and more	Java, JavaScript / TypeScript, Python, C#

Tools	Needs extra libraries for reports	Trace viewer, video, screenshots, codegen built in
Network control	Limited	Easy: intercept, fake, wait for responses
Mobile apps	With Appium	Only mobile web emulation
Community	Oldest and biggest, most jobs	Newer, growing very fast

---

13. Quick interview answers

- IMPORTANT **What is Playwright?**
  An open-source library from Microsoft to automate Chromium, Firefox and WebKit. It supports Java, JavaScript / TypeScript, Python and C#.
- IMPORTANT **Explain Playwright, Browser, BrowserContext and Page.**
  Playwright starts everything. Browser is the browser program. BrowserContext is an isolated profile like an incognito window. Page is one tab.
- IMPORTANT **Why is Playwright less flaky than Selenium?**
  Auto-waiting before each action, and web-first assertions that retry until true.
- IMPORTANT **What is auto-waiting?**
  Before an action, Playwright waits until the element is attached, visible, stable, enabled and not covered.
- IMPORTANT **What is a Locator? Why is it better than a WebElement?**
  A Locator is a recipe to find an element, and it searches again on every use, so it never goes stale.
- IMPORTANT **Which locators do you prefer?**
  getByRole, getByLabel, getByText, getByTestId first, because they match what users see. CSS and XPath when needed.
- IMPORTANT **What is strict mode?**
  If a locator matches more than one element, an action throws an error instead of picking one. Fix with a better locator, first(), or nth().
- IMPORTANT **fill() vs pressSequentially()?**
  fill() sets the whole value at once. pressSequentially() types key by key like a user, and respects maxlength.

- IMPORTANT **How do you handle alerts?**
  Register `page.onceDialog(...)` BEFORE the action. Without a handler, Playwright dismisses dialogs automatically.
- IMPORTANT **How do you handle frames and new tabs?**
  `page.frameLocator("#id")` for frames. `page.waitForPopup(() → click)` for a new tab.
- IMPORTANT **Why a new BrowserContext per test?**
  Clean cookies and storage for every test, so tests are independent. Creating a context is very fast.
- IMPORTANT **Selenium vs Playwright?**
  See the table above: auto-wait, retrying assertions, speed, built-in trace and video, contexts.
- GOOD TO KNOW **How do you debug a failing test?**
  Run headed with slowMo, use `page.pause()` and the Inspector, and open the trace viewer for failures in CI.
- GOOD TO KNOW **How do you test the same code on many browsers?**
  Change `chromium()` to `firefox()` or `webkit()`, or loop over the three.
- GOOD TO KNOW **How do you reuse a login?**
  Log in once, save `context.storageState(...)` to a file, and start other contexts from that file.
- GOOD TO KNOW **How do you mock an API response?**
  `page.route("**/api/users", route → route.fulfill(...))`.
- GOOD TO KNOW **Can Playwright test APIs?**
  Yes, with `playwright.request().newContext()` you can send GET, POST and so on, and check `response.status()`.
- GOOD TO KNOW **Does Playwright support parallel tests?**
  Yes. With TestNG you can run classes in parallel, and because each test uses its own context they do not disturb each other.